

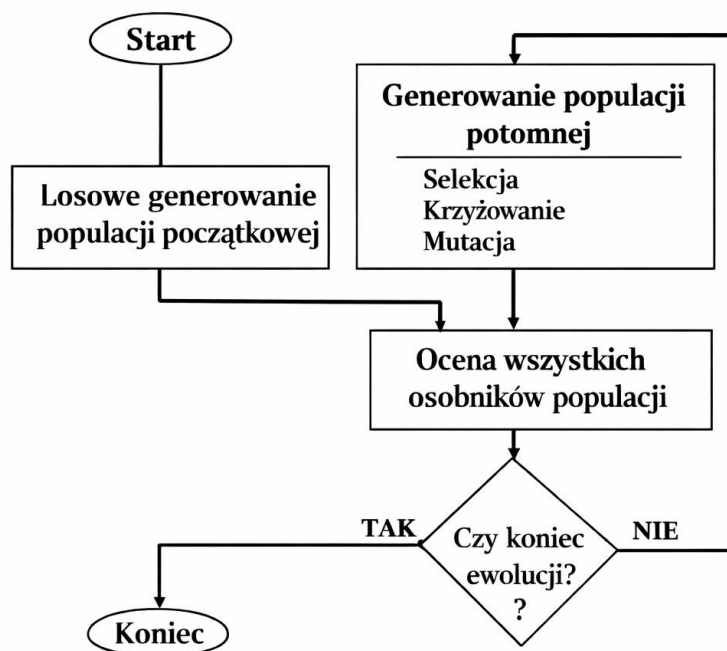
Algorytmy genetyczne

Wprowadzenie do algorytmów genetycznych

Współczesne problemy optymalizacyjne, występujące w inżynierii, informatyce, ekonomii czy logistyce, coraz częściej charakteryzują się dużą złożonością obliczeniową, nieliniowością oraz obecnością wielu lokalnych ekstremów. Klasyczne metody deterministyczne, oparte na rachunku różniczkowym lub przeszukiwaniu zupełnym, w wielu przypadkach okazują się niewystarczające lub zbyt kosztowne obliczeniowo. W odpowiedzi na te ograniczenia rozwinięto metody heurystyczne i metaheurystyczne, wśród których istotne miejsce zajmują algorytmy genetyczne.

Algorytmy genetyczne to metaheurystyczne algorytmy optymalizacyjne, inspirowane mechanizmami ewolucji biologicznej opisanymi przez Charlesa Darwina. Wykorzystują pojęcia takie jak selekcja naturalna, krzyżowanie i mutacja, aby znajdować dobre (często bliskie optymalnym) rozwiązania złożonych problemów.

Jak działa algorytm genetyczny?



Rysunek 1 Schemat blokowy algorytmu genetycznego

1. Inicjalizacja populacji
Tworzymy losowy zbiór rozwiązań (np. 100 chromosomów).
2. Ocena (funkcja przystosowania – fitness)
Każdemu rozwiązaniu przypisujemy wartość określającą jego jakość.
3. Selekcja
Wybieramy najlepsze osobniki (np. metoda ruletki, turniejowa).
4. Krzyżowanie (crossover)
Łączymy dwa rozwiązania, tworząc nowe potomstwo.
5. Mutacja
Losowo zmieniamy niektóre geny, aby zwiększyć różnorodność.
6. Zastąpienie populacji
Nowe pokolenie zastępuje stare.

Proces powtarzamy aż do:

- osiągnięcia określonej liczby pokoleń,
- znalezienia wystarczająco dobrego rozwiązania,
- spełnienia warunku stopu.

Ocena przystosowania

Ocena przystosowania (ang. *fitness evaluation*) to etap algorytmu genetycznego, w którym każdemu osobnikowi przypisuje się wartość liczbowa opisującą jakość jego rozwiązania. Na jej podstawie później działa selekcja (np. ruletka), krzyżowanie i utrzymanie najlepszych osobników.

Co jest oceniane?

Osobnik reprezentuje rozwiązanie:

- genotyp – zakodowana postać (np. ciąg bitów),
- fenotyp – rozwiązanie po dekodowaniu (np. wektor x).

Ocena przystosowania polega na obliczeniu:

$$f_i = f(x_i)$$

gdzie:

- x_i – fenotyp osobnika i ,

- $f(\cdot)$ – funkcja celu / funkcja przystosowania,
- f_i – przystosowanie (fitness).

Funkcja przystosowania a funkcja celu

Najczęściej fitness bazuje na funkcji celu, ale bywa przekształcany, żeby:

- mieć zawsze wartości dodatnie,
- pasować do typu zadania (maksymalizacja/minimalizacja),
- nie powodować zbyt dużej presji selekcyjnej.

Gdy problem jest maksymalizacyjny

Najprościej:

$$\text{fitness}(x) = f(x)$$

Gdy problem jest minimalizacyjny

Często przekształca się do postaci maksymalizacyjnej, np.:

$$\text{fitness}(x) = \frac{1}{1 + f(x)}$$

albo (jeśli znamy $f_{\max}$ w populacji):

$$\text{fitness}(x) = f_{\max} - f(x)$$

Przystosowanie bezwzględne i względne

W praktyce rozróżnia się:

- przystosowanie bezwzględne: f_i (wynik funkcji fitness),
- przystosowanie względne (do selekcji proporcjonalnej):

$$p_i = \frac{f_i}{\sum_{j=1}^N f_j}$$

To właśnie p_i wyznacza „udział” osobnika w kole ruletki.

Dodatkowo często liczy się dystrybuantę (sumę skumulowaną):

$$F_i = \sum_{k=1}^i p_k$$

żeby wylosować osobnika na podstawie liczby $r \in [0,1]$.

Na co trzeba uważać przy ocenie?

- Skalowanie: jeśli jeden osobnik ma fitness dużo większy od reszty, selekcja może zbyt szybko zdominować populację (przedwczesna zbieżność).
- Ograniczenia: w problemach z ograniczeniami stosuje się kary, np.:

$$\text{fitness}(x) = f(x) - \lambda \cdot \text{penalty}(x)$$

- Koszt obliczeniowy: ocena fitness bywa najdroższą częścią algorytmu (np. symulacje, modele fizyczne), więc stosuje się cache, aproksymacje lub zrównoleglenie.

Uwaga:

Przekształcenie funkcji zysku do funkcji przystosowania jest wykonywane w etapie oceny populacji, bezpośrednio po obliczeniu wartości funkcji celu, a przed selekcją.

Dlaczego wykonuje się to przekształcenie?

W metodzie ruletki:

$$p_i = \frac{f(x_i)}{\sum f(x_j)}$$

wartości muszą być nieujemne, bo reprezentują prawdopodobieństwa.

Jeśli funkcja zysku przyjmuje wartości ujemne, to:

- nie można bezpośrednio użyć jej w ruletce,
- mogłyby powstać ujemne prawdopodobieństwa.

Dana jest funkcja zysku:

$$g(x) \in \mathbb{R}$$

Może ona przyjmować dowolne wartości rzeczywiste (także ujemne).

Aby zastosować selekcję proporcjonalną, potrzebujemy funkcji przystosowania:

$$f(x) = g(x) - C_{\min}$$

Rola $C_{\min}$

$$C_{\min}$$

to liczba nie większa niż najmniejsza wartość funkcji w aktualnej populacji:

$$C_{\min} \leq \min_i g(x_i)$$

Najczęściej przyjmuje się:

$$C_{\min} = \min_i g(x_i)$$

Wtedy:

$$f(x_i) = g(x_i) - \min_j g(x_j)$$

co gwarantuje:

$$f(x_i) \geq 0$$

Najgorszy osobnik ma wtedy przystosowanie równe 0.

Co to oznacza?

To jest przesunięcie funkcji w górę, tak aby:

- wszystkie wartości były nieujemne,
- zachowane zostały proporcje między osobnikami,
- można było zastosować selekcję proporcjonalną.

Przykład liczbowy

Założmy:

$$g(x) = [-5, 2, 7]$$

Minimum:

$$C_{\min} = -5$$

Nowa funkcja przystosowania:

$$f(x) = g(x) - (-5)$$

$$f(x) = [0, 7, 12]$$

Teraz można bezpiecznie policzyć prawdopodobieństwa.

Co w przypadku, gdy optymalizujemy funkcję kosztu, czyli rozwiązujemy problem minimalizacji?

Dana jest funkcja kosztu:

$$u(x) \in \mathbb{R}$$

Chcemy ją zminimalizować.

Algorytm genetyczny (zwłaszcza selekcja proporcjonalna) zwykle działa w trybie maksymalizacji — większe fitness = większa szansa wyboru.

Dlatego przekształcamy funkcję kosztu do funkcji przystosowania:

$$f(x) = C_{\max} - u(x)$$

Co to oznacza?

- im mniejszy koszt $u(x)$,
- tym większe przystosowanie $f(x)$,
- tym większe prawdopodobieństwo selekcji.

Czyli odwracamy problem minimalizacji do postaci maksymalizacyjnej.

Rola $C_{\max}$

$$C_{\max}$$

to liczba nie mniejsza niż maksymalna wartość funkcji kosztu w aktualnej populacji:

$$C_{\max} \geq \max_i u(x_i)$$

Najczęściej przyjmuje się:

$$C_{\max} = \max_i u(x_i)$$

Wtedy:

$$f(x_i) = \max_j u(x_j) - u(x_i)$$

co gwarantuje:

$$f(x_i) \geq 0$$

Najgorszy osobnik ma fitness równy 0.

Przykład liczbowy

Założmy:

$$u(x) = [10, 4, 7]$$

Maksimum:

$$C_{\max} = 10$$

Fitness:

$$f(x) = [0, 6, 3]$$

Osobnik z kosztem 4 (najlepszy) ma największe przystosowanie.

Tabela 1 Ocena przystosowania funkcji kosztu, zysku.

Funkcja zysku (maksymalizacja)	Funkcja kosztu (minimalizacja)
$f(x) = g(x) - C_{\min}$	$f(x) = C_{\max} - u(x)$
Przesunięcie w górę	Odbicie względem maksimum
Usuwa wartości ujemne	Zamienia min na max

Metody selekcji osobników

W literaturze dotyczącej algorytmów ewolucyjnych opisano wiele metod selekcji osobników, których celem jest wybór rozwiązań najlepiej przystosowanych do przekazania swoich cech kolejnemu pokoleniu. Do najczęściej stosowanych i klasycznych podejść należą:

1. Metoda koła ruletki (klasyczna)

Prawdopodobieństwo wyboru osobnika jest proporcjonalne do wartości jego funkcji przystosowania. Im lepsze przystosowanie, tym większa „część koła ruletki” przypisana danemu osobnikowi, a tym samym większa szansa na selekcję.

2. Metoda rankingowa

Osobniki są sortowane według wartości przystosowania, a następnie przypisuje się im prawdopodobieństwo wyboru na podstawie pozycji w rankingu, a nie bezpośrednio wartości funkcji celu. Ogranicza to dominację bardzo silnych osobników.

3. Metoda turniejowa

Losowo wybiera się niewielką grupę osobników (turniej), z której do dalszego etapu przechodzi najlepszy. Wielkość turnieju wpływa na siłę presji selekcyjnej.

4. Metoda De Jonga

Zakłada modyfikację klasycznej selekcji proporcjonalnej poprzez skalowanie wartości funkcji przystosowania, co pozwala kontrolować presję selekcyjną i zapobiegać przedwczesnej zbieżności.

5. Metoda Brindle'a

Opiera się na modyfikacjach selekcji proporcjonalnej i mechanizmach ograniczających nadmierne uprzywilejowanie najlepszych osobników, wspierając zachowanie różnorodności populacji.

6. Metoda Mauldina

Wprowadza mechanizmy utrzymania różnorodności genetycznej w populacji, przeciwdziałając szybkiemu zdominowaniu populacji przez podobne rozwiązania.

7. Metoda Michalewicza

Uwzględnia adaptacyjne mechanizmy selekcji, często powiązane z dynamiczną zmianą parametrów, co pozwala lepiej dostosować proces selekcji do etapu ewolucji.

Tabela 2 Tabela porównawcza metod selekcji w algorytmach genetycznych

Metoda	Prawdopodobieństwo wyboru (p_i)	Parametry sterujące	Charakterystyka presji selekcyjnej
Koła ruletki	$p_i = \frac{f_i}{\sum_{j=1}^N f_j}$	brak	Proporcjonalna do wartości funkcji przystosowania
Rankingowa (liniowa)	$p_i = \frac{1}{N} \left[2 - s + 2(r_i - 1) \frac{s-1}{N-1} \right]$	$s \in [1,2]$	Kontrolowana, niezależna od bezwzględnych wartości (f_i)
Turniejowa (k-osobowa)	$p_i = \left(\frac{n_i}{N} \right)^k - \left(\frac{n_i - 1}{N} \right)^k$	k – rozmiar turnieju	Rośnie wraz z (k); łatwa kontrola presji
De Jonga (skalowanie liniowe)	$p_i = \frac{af_i + b}{\sum_{j=1}^N (af_j + b)}$	(a, b)	Regulowana przez skalowanie funkcji przystosowania
Brindle'a	$p_i = \frac{f_i}{\sum f_j}$ dla ($f_i \geq f_{min}$), w przeciwnym razie 0	f_{min}	Ograniczenie dominacji słabych osobników
Mauldina (sharing)	$p_i = \frac{f_i / \sum_j sh(d_{ij})}{\sum_k f_k / \sum_j sh(d_{kj})}$	σ_{share}	Utrzymanie różnorodności populacji
Michalewicza (adaptacyjna)	$p_i = \frac{f_i^\beta}{\sum_{j=1}^N f_j^\beta}$	β (zmienne w czasie)	Adaptacyjnie wzmacniana lub osłabiana presja

Oznaczenia:

- f_i – wartość funkcji przystosowania osobnika i
- N – liczebność populacji
- r_i – pozycja w rankingu
- n_i – liczba osobników nie gorszych od i
- k – rozmiar turnieju
- $sh(d_{ij})$ – funkcja dzielenia niszy
- β – parametr adaptacyjny

Przykład użycia metody proporcjonalnej

Poniższa tabela przedstawia populację sześciu osobników w algorytmie genetycznym wraz z ich genotypami, odpowiadającymi im fenotypami oraz wartościami przystosowania bezwzględnego i względnego.

Tabela 3 Genotyp, fenotyp i przystosowanie osobnika.

Lp.	genotyp	fenotyp	przystosowanie		dystribuanta
			bezwzględne	względne	
1	[1011 011 00101] ^T	[11 3 5] ^T	1323	0.1949	0.1949
2	[0111 010 11110] ^T	[7 2 30] ^T	1640	0.2413	0.2413
3	[1001 101 111100] ^T	[17 5 4] ^T	1321	0.1947	0.4362
4	[0001 011 11101] ^T	[1 3 29] ^T	993	0.1463	0.7773
5	[1100 000 11010] ^T	[12 0 26] ^T	1432	0.2109	1.0000
6	[1100 000 11010] ^T	[12 0 26] ^T	1432	0.2109	1.0000

$$\sum_{i=1}^5 f(x_i) = 6709$$

W dalszej części omawiamy poszczególne fazy użycia metody proporcjonalnej.

1. Dane wejściowe przystosowania bezwzględnego:

$$f(x_1) = 1323$$

$$f(x_2) = 1640$$

$$f(x_3) = 1321$$

$$f(x_4) = 993$$

$$f(x_5) = 1432$$

2. Obliczenie sumy przystosowań

$$\sum_{i=1}^5 f(x_i) = 1323 + 1640 + 1321 + 993 + 1432$$

$$\sum f(x_i) = 6709$$

3. Obliczenie przystosowania względnego

W metodzie proporcjonalnej prawdopodobieństwo wyboru osobnika wynosi:

$$p_i = \frac{f(x_i)}{\sum_{j=1}^N f(x_j)}$$

Osobnik 1

$$p_1 = \frac{1323}{6709} = 0.1949$$

Osobnik 2

$$p_2 = \frac{1640}{6709} = 0.2413$$

Osobnik 3

$$p_3 = \frac{1321}{6709} = 0.1969 \approx 0.1947$$

Osobnik 4

$$p_4 = \frac{993}{6709} = 0.1480 \approx 0.1463$$

Osobnik 5

$$p_5 = \frac{1432}{6709} = 0.2134 \approx 0.2109$$

4. Obliczenie dystrybuanty (sumy skumulowanej)

Dystrybuanta to skumulowana suma prawdopodobieństw:

$$F_i = \sum_{k=1}^i p_k$$

$$F_1 = 0.1949$$

$$F_2 = 0.1949 + 0.2413 = 0.4362$$

$$F_3 = 0.4362 + 0.1947 = 0.6309$$

$$F_4 = 0.6309 + 0.1463 = 0.7772$$

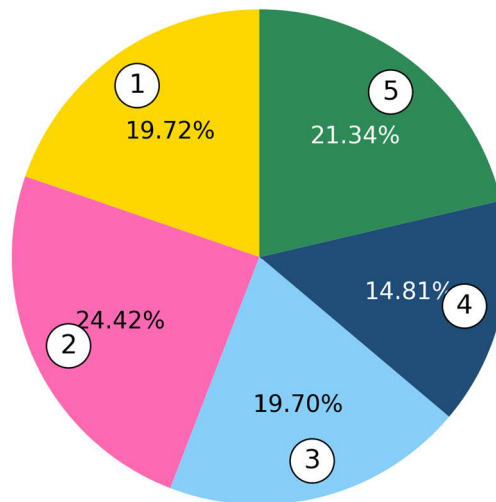
$$F_5 = 0.7772 + 0.2109 = 1.0000$$

5. Interpretacja w metodzie koła ruletki

Losujemy liczbę $r \in [0,1]$:

- jeśli $r \in [0,0.1949) \rightarrow$ wybieramy osobnika 1
- jeśli $r \in [0.1949,0.4362) \rightarrow$ osobnik 2
- itd.

Każdy osobnik otrzymuje „wycinek koła” proporcjonalny do swojego przystosowania.



Rysunek 2 Koło ruletki w omawianym przykładzie

Uwagi: Skąd pochodzą wartości przystosowania bezwzględnego?

Wartości przystosowania bezwzględnego $f(x_i)$ są wynikiem zastosowania funkcji celu (funkcji przystosowania) do fenotypu danego osobnika.

Formalnie:

$$f(x_i) = F(\varphi(g_i))$$

gdzie:

- g_i – genotyp,
- $\varphi(g_i)$ – dekodowany fenotyp,
- $F(\cdot)$ – funkcja celu,
- $f(x_i)$ – przystosowanie bezwzględne.

Co to oznacza w praktyce?

Wartości 1323, 1640, 1321 itd. nie są losowe.

Są one wynikiem:

1. Dekodowania genotypu do fenotypu,
2. Podstawienia fenotypu do funkcji celu.

Przykład mechanizmu obliczeń

Założmy, że:

Fenotyp:

$$x = [11, 3, 5]$$

Funkcja celu:

$$F(x) = x_1^2 + 3x_2 + 5x_3$$

Wtedy:

$$\begin{aligned} F([11, 3, 5]) &= 11^2 + 3 \cdot 3 + 5 \cdot 5 \\ &= 121 + 9 + 25 \\ &= 155 \end{aligned}$$

Kluczowa zasada

Przystosowanie bezwzględne zawsze wynika z:

$$f(x_i) = F(x_i)$$

czyli bezpośredniej oceny rozwiązania przez funkcję optymalizowaną.

W zależności od problemu:

Problem maksymalizacji

$$f(x) = F(x)$$

Problem minimalizacji

Często stosuje się przekształcenie:

$$f(x) = \frac{1}{1 + F(x)}$$

lub

$$f(x) = F_{max} - F(x)$$

aby móc stosować selekcję proporcjonalną.

Operatory genetyczne

Operatory genetyczne to podstawowe mechanizmy wykorzystywane w algorytmach genetycznych do przekształcania populacji rozwiązań w kolejnych pokoleniach. Odpowiadają one za tworzenie nowych osobników oraz za przekazywanie i modyfikację informacji genetycznej (czyli reprezentacji rozwiązania).

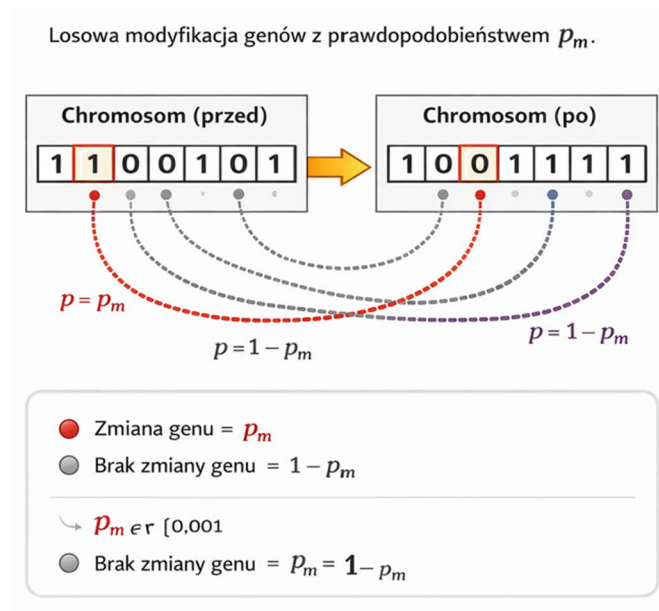
Stanowią one odpowiednik procesów biologicznych, takich jak krzyżowanie i mutacja.

Operator mutacji w algorytmie genetycznym

Mutacja to operator genetyczny polegający na losowej modyfikacji jednego lub kilku genów w chromosomie osobnika. Jej głównym celem jest wprowadzanie różnorodności genetycznej do populacji oraz zapobieganie przedwczesnej zbieżności algorytmu do rozwiązania lokalnie optymalnego.

Ze względu na to, który element organizmu lub rozwiązania podlega zmianie, mutacje można podzielić na dwie podstawowe kategorie:

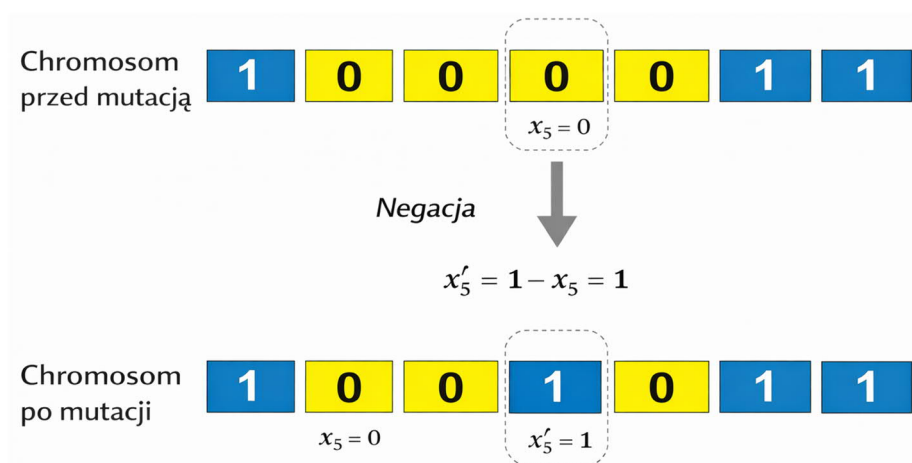
- mutacje genotypowe (genetyczne), polegające na modyfikacji materiału genetycznego, czyli bezpośredniej zmianie genotypu,
- mutacje fenotypowe (parametryczne), odnoszące się do zmian na poziomie cech obserwowalnych, bez ingerencji w sam genotyp.



Rysunek 3 Interpretacja graficzna mutacji

Inne mechanizmy mutacji:

- inwersja (ang. inversion),
- wstawianie (ang. insertion),
- przenoszenie (ang. displacement),
- wzajemna wymiana (ang. reciprocal exchange).



Rysunek 4 Przykład mutacji jednopunktowej bitu

Operator krzyżowania w algorytmie genetycznym

Krzyżowanie (ang. *crossover*) jest operatorem w algorytmach genetycznych analogicznym do biologicznej rekombinacji zachodzącej pomiędzy nićmi DNA chromosomów z par homologicznych. W algorytmach genetycznych proces ten polega na wymianie fragmentów materiału genetycznego, czyli części chromosomu, pomiędzy dwoma osobnikami wybranymi jako rodzice. W wyniku tej operacji powstaje zazwyczaj dwóch potomków, którzy stanowią nowe, potencjalnie lepsze rozwiązania rozpatrywanego problemu.

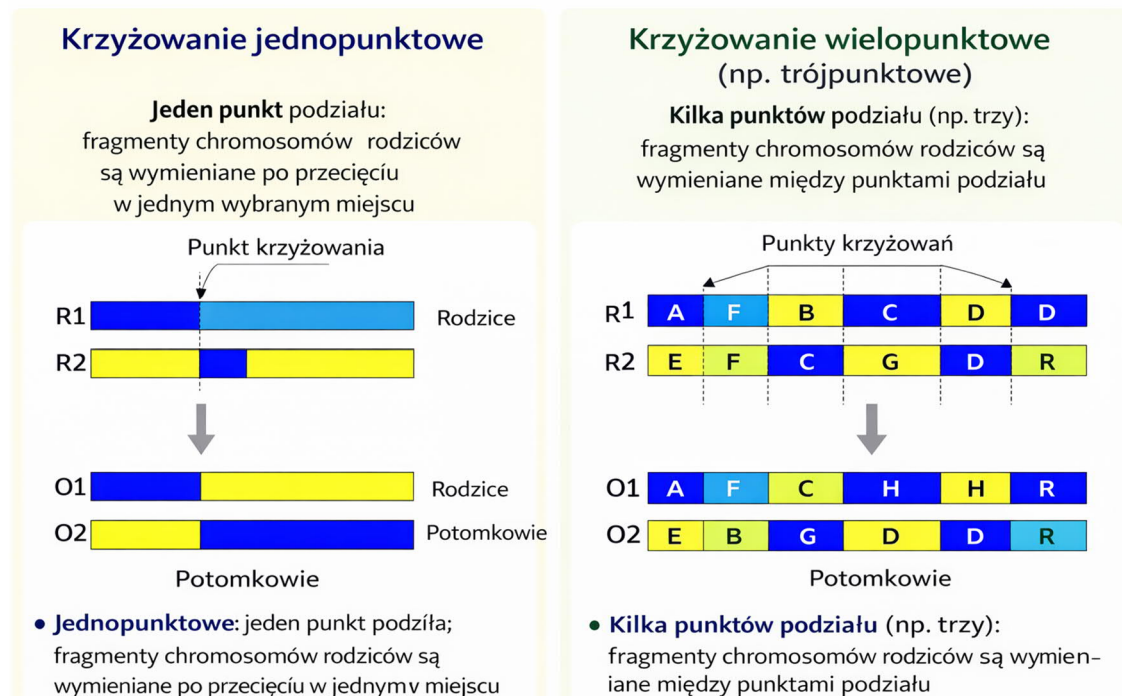
Krzyżowanie przebiega w dwóch etapach:

1. Kojarzenie w sposób losowy osobników z puli rodzicielskiej w pary,
2. Każda para rodziców przechodzi proces wymiany materiału genetycznego (tj. wymiany fragmentów sekwencji kodowych pomiędzy chromosomami osobników rodzicielskich).

Krzyżowanie zachodzi z pewnym prawdopodobieństwem nazywanym **prawdopodobieństwem krzyżowania**.

Najczęściej stosowanym sposobem krzyżowania jest:

- krzyżowanie jednopunktowe,
- krzyżowanie wielopunktowe,
- krzyżowanie jednorodne.

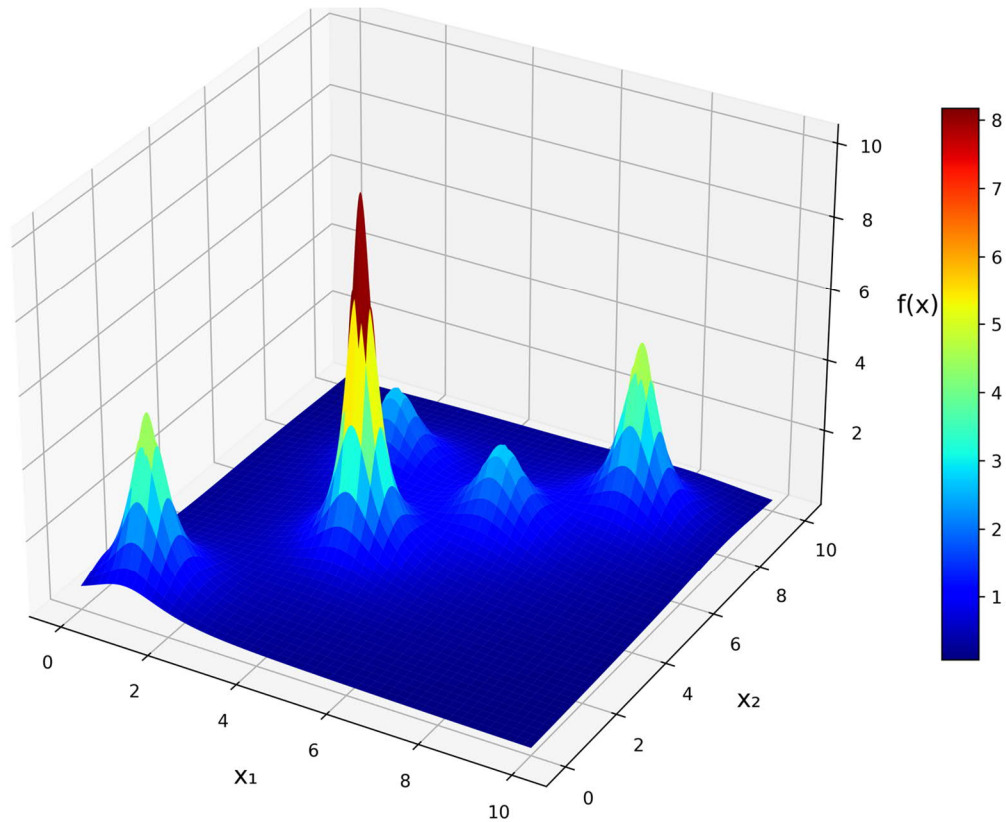


Rysunek 5 Podstawowe sposoby krzyżowania

Przykład 1

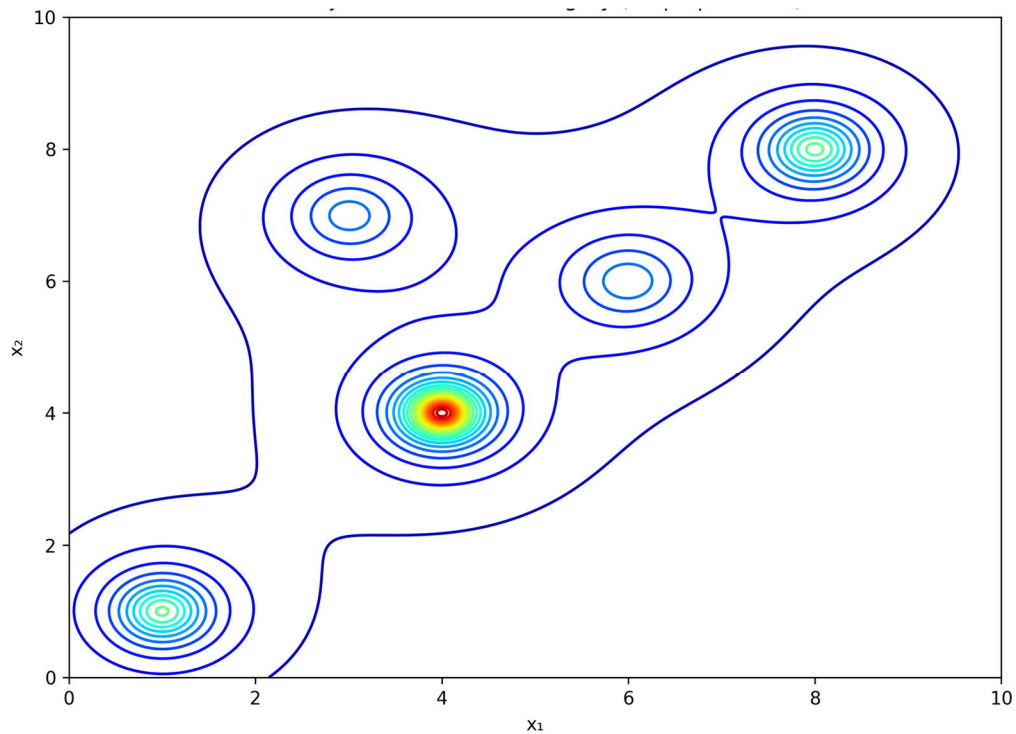
Cel programu

Celem programu jest znalezienie maksimum globalnego funkcji Shekel-5 przy wykorzystaniu algorytmu genetycznego (GA).



Rysunek 6 Wykres funkcji Shekel-5

Funkcja Shekel-5 jest funkcją wielomodalną, posiadającą kilka maksimum lokalnych oraz jedno maksimum globalne, co czyni ją dobrym testem skuteczności metod optymalizacji globalnej.



Rysunek 7 Funkcji Shekel-5 - widok z góry (mapa poziomnic)

Program dodatkowo umożliwia wizualizację procesu optymalizacji w postaci animacji, pokazując:

- rozkład populacji w przestrzeni zmiennych,
- ewolucję najlepszego rozwiązania,
- zbieżność algorytmu w czasie.

Co robi program?

Program realizuje klasyczny schemat algorytmu genetycznego:

Inicjalizacja

- Losowe wygenerowanie początkowej populacji osobników w przedziale:

$$x_1, x_2 \in [0,10]$$

Ocena przystosowania

- Obliczenie wartości funkcji Shekel-5 dla każdego osobnika:

$$f(x) = \sum_{i=1}^5 \frac{1}{c_i + \|x - a_i\|^2}$$

Selekcja

- Selekcja turniejowa (wybór najlepszych osobników z losowej grupy).

Krzyżowanie

- Krzyżowanie arytmetyczne (liniowa kombinacja rodziców).

Mutacja

- Mutacja Gaussowska (dodanie losowego zaburzenia do genów).

Elityzm

- Najlepsi osobnicy przechodzą bez zmian do następnej generacji.

Animacja

W każdej generacji:

- aktualizowana jest pozycja osobników na mapie poziomic,
- zaznaczany jest najlepszy osobnik,
- aktualizowany jest wykres najlepszego fitness w czasie.

Parametry algorytmu

Program pozwala regulować następujące parametry:

Parametr	Znaczenie
pop_size	Liczba osobników w populacji
generations	Liczba generacji (iteracji)
crossover_rate	Prawdopodobieństwo krzyżowania
mutation_rate	Prawdopodobieństwo mutacji genu
mutation_sigma	Odchylenie standardowe mutacji

Parametr	Znaczenie
tournament_k	Rozmiar turnieju selekcyjnego
elitism	Liczba najlepszych osobników kopiowanych bez zmian
seed	Ziarno generatora losowego
save_mp4	Czy zapisać animację do pliku

Wyniki programu

Program generuje:

Najlepsze rozwiązanie

- Współrzędne:

$$x^* = (x_1^*, x_2^*)$$

- Wartość funkcji:

$$f(x^*)$$

Animację procesu optymalizacji

- Lewy panel: ruch populacji w przestrzeni zmiennych.
- Prawy panel: wykres zbieżności (najlepszy fitness w czasie).

Obserwacje typowe

- Początkowo populacja jest rozproszona.
- Z czasem osobniki koncentrują się wokół maksimum globalnego.
- Wykres fitness stabilizuje się przy wartości maksymalnej.

Listing kodu w Python

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
```

```

# -----
# Funkcja Shekel-5 (2D)
# -----
def shekel5(x: np.ndarray) -> float:
    """
    Shekel-5 w 2D (wariant demonstracyjny).
    Maksymalizacja:  $f(x) = \sum 1 / (c_i + ||x - a_i||^2)$ 
    """
    c = np.array([0.1, 0.2, 0.2, 0.4, 0.4], dtype=float)
    a = np.array([
        [4.0, 4.0],
        [1.0, 1.0],
        [8.0, 8.0],
        [6.0, 6.0],
        [3.0, 7.0],
    ], dtype=float)

    diff = a - x # (5,2)
    sq = np.sum(diff * diff, axis=1)
    return float(np.sum(1.0 / (c + sq)))

def evaluate_population(pop: np.ndarray) -> np.ndarray:
    return np.array([shekel5(ind) for ind in pop], dtype=float)

# -----
# Operatory GA
# -----
def init_population(pop_size: int, dim: int, bounds: np.ndarray, rng:
np.random.Generator) -> np.ndarray:
    lo = bounds[:, 0]
    hi = bounds[:, 1]
    return rng.uniform(lo, hi, size=(pop_size, dim))

def tournament_selection(pop: np.ndarray, fitness: np.ndarray, k: int,
rng: np.random.Generator) -> np.ndarray:
    idx = rng.integers(0, len(pop), size=k)
    best = idx[np.argmax(fitness[idx])]
    return pop[best].copy()

def arithmetic_crossover(p1: np.ndarray, p2: np.ndarray, rng:
np.random.Generator) -> tuple[np.ndarray, np.ndarray]:
    alpha = rng.uniform(0.0, 1.0)
    c1 = alpha * p1 + (1.0 - alpha) * p2
    c2 = alpha * p2 + (1.0 - alpha) * p1

```

```

        return c1, c2

def gaussian_mutation(x: np.ndarray, pm: float, sigma: float, bounds:
np.ndarray, rng: np.random.Generator) -> np.ndarray:
    y = x.copy()
    mask = rng.random(size=y.shape) < pm
    if np.any(mask):
        y[mask] += rng.normal(0.0, sigma, size=np.count_nonzero(mask))
    y = np.clip(y, bounds[:, 0], bounds[:, 1])
    return y

def step_ga(
    pop: np.ndarray,
    fitness: np.ndarray,
    bounds: np.ndarray,
    rng: np.random.Generator,
    crossover_rate: float,
    mutation_rate: float,
    mutation_sigma: float,
    tournament_k: int,
    elitism: int,
) -> tuple[np.ndarray, np.ndarray]:
    """Wykonuje jedną generację GA (maksymalizacja)."""
    pop_size = len(pop)

    # sort malejaco po fitness
    order = np.argsort(fitness)[::-1]
    pop = pop[order]
    fitness = fitness[order]

    new_pop = []
    # elityzm
    for i in range(elitism):
        new_pop.append(pop[i].copy())

    # reszta populacji
    while len(new_pop) < pop_size:
        p1 = tournament_selection(pop, fitness, tournament_k, rng)
        p2 = tournament_selection(pop, fitness, tournament_k, rng)

        if rng.random() < crossover_rate:
            c1, c2 = arithmetic_crossover(p1, p2, rng)
        else:
            c1, c2 = p1.copy(), p2.copy()

        c1 = gaussian_mutation(c1, mutation_rate, mutation_sigma,
bounds, rng)

```

```

        new_pop.append(c1)

        if len(new_pop) < pop_size:
            c2 = gaussian_mutation(c2, mutation_rate, mutation_sigma,
bounds, rng)
            new_pop.append(c2)

    new_pop = np.array(new_pop, dtype=float)
    new_fit = evaluate_population(new_pop)
    return new_pop, new_fit

# -----
# Wizualizacja: tło (poziomice) + animacja populacji
# -----
def precompute_contours(bounds: np.ndarray, grid_n: int = 250):
    x = np.linspace(bounds[0, 0], bounds[0, 1], grid_n)
    y = np.linspace(bounds[1, 0], bounds[1, 1], grid_n)
    X, Y = np.meshgrid(x, y)
    Z = np.zeros_like(X, dtype=float)
    for i in range(grid_n):
        for j in range(grid_n):
            Z[i, j] = shekel5(np.array([X[i, j], Y[i, j]],
dtype=float))
    return X, Y, Z

def animate_ga_shekel5(
    pop_size: int = 100,
    generations: int = 200,
    crossover_rate: float = 0.9,
    mutation_rate: float = 0.15,
    mutation_sigma: float = 0.30,
    tournament_k: int = 3,
    elitism: int = 2,
    seed: int = 7,
    save_mp4: bool = False,
    out_path: str = "ga_shekel5.mp4",
):
    rng = np.random.default_rng(seed)
    bounds = np.array([[0.0, 10.0],
                        [0.0, 10.0]], dtype=float)

    # start
    pop = init_population(pop_size, dim=2, bounds=bounds, rng=rng)
    fitness = evaluate_population(pop)

    # tło (mapa poziomic)
    X, Y, Z = precompute_contours(bounds, grid_n=220)

```

```

# historia najlepszych wartości
best_hist = []

# figura: lewy panel (poziomice + populacja), prawy panel (best
fitness)
fig = plt.figure(figsize=(11, 5), dpi=140)
gs = fig.add_gridspec(1, 2, width_ratios=[1.2, 1.0])

ax0 = fig.add_subplot(gs[0, 0])
ax1 = fig.add_subplot(gs[0, 1])

# Poziomice (styl podobny do klasycznych wykresów)
ax0.contour(X, Y, Z, levels=20)
ax0.set_xlabel("x1")
ax0.set_ylabel("x2")
ax0.set_title("Shekel-5: populacja GA (widok z góry)")

# Punkty populacji + najlepszy
scat = ax0.scatter(pop[:, 0], pop[:, 1], s=18)
best_dot = ax0.scatter([], [], s=70) # podmienimy w update

# Wykres best fitness
ax1.set_title("Najlepszy fitness w czasie")
ax1.set_xlabel("Generacja")
ax1.set_ylabel("fitness")
line_best, = ax1.plot([], [])
ax1.set_xlim(0, generations)
# ustawimy ylim dynamicznie na podstawie pierwszych wartości
ax1.set_ylim(0, max(1.0, float(np.max(fitness)) * 1.05))

info = ax0.text(
    0.02, 0.98, "", transform=ax0.transAxes,
    ha="left", va="top"
)

def update(frame: int):
    nonlocal pop, fitness

    # sort i pobierz best przed wykonaniem kroku (dla czytelności)
    best_idx = int(np.argmax(fitness))
    best_x = pop[best_idx]
    best_f = float(fitness[best_idx])
    best_hist.append(best_f)

    # aktualizuj scatter (populacja) i best punkt
    scat.set_offsets(pop)
    best_dot.set_offsets([best_x[0], best_x[1]])

```

```

        # aktualizuj linię best fitness
        xs = np.arange(len(best_hist))
        line_best.set_data(xs, best_hist)

        # dopasuj ylim jeśli rośnie
        current_top = ax1.get_ylim()[1]
        if best_f * 1.05 > current_top:
            ax1.set_ylim(ax1.get_ylim()[0], best_f * 1.10)

        info.set_text(f"Gen: {frame:3d} | best f = {best_f:.4f} | best
x = [{best_x[0]:.3f}, {best_x[1]:.3f}]")

        # wykonaj krok GA na następną generację (oprócz ostatniej
klatki)
        if frame < generations - 1:
            pop, fitness = step_ga(
                pop, fitness, bounds, rng,
                crossover_rate, mutation_rate, mutation_sigma,
                tournament_k, elitism
            )

            return scat, best_dot, line_best, info

    anim = FuncAnimation(fig, update, frames=generations, interval=80,
blit=False, repeat=False)
    plt.tight_layout()

    if save_mp4:
        # Wymaga: ffmpeg w systemie
        anim.save(out_path, fps=12)
        print(f"Zapisano animację do: {out_path}")

    plt.show()

if __name__ == "__main__":
    animate_ga_shekel5(
        pop_size=120,
        generations=220,
        crossover_rate=0.9,
        mutation_rate=0.15,
        mutation_sigma=0.30,
        tournament_k=3,
        elitism=2,
        seed=7,
        save_mp4=False,    # ustaw na True jeśli chcesz zapisać mp4
        out_path="ga_shekel5.mp4",
    )

```


Podstawowe pojęcia

1. Osobnik (Individual)

Osobnik to pojedyncze rozwiązanie problemu. W przykładzie OneMax jest to ciąg 0 i 1 o długości 20. Każdy osobnik posiada chromosom, wartość fitness (siła), numer dentyfikacyjny (cargo) oraz licznik określający ile razy był rodzicem.

2. Chromosom

Chromosom to reprezentacja rozwiązania, czyli wektor genów.

3. Gen

Gen to najmniejsza jednostka informacji w chromosomie. W naszym przypadku genem jest pojedyncza wartość 0 lub 1.

4. Populacja

Populacja to zbiór wszystkich osobników w danej generacji. W przykładzie składa się z 20 osobników.

5. Generacja

Generacja to jedno pokolenie populacji. Powstaje poprzez selekcję, krzyżowanie oraz mutację osobników z poprzedniej generacji. Każda generacja jest wynikiem jednej pełnej iteracji algorytmu.

6. Genotyp

Genotyp - struktura złożona z chromosomów, zbiór informacji opisujący osobnika,

7. Fenotyp

Fenotyp - zbiór konkretnych wartości odpowiadających danemu genotypowi, posiada go każdy osobnik,

8. Iteracja

Iteracja to jedno wykonanie pętli algorytmu genetycznego. W praktyce w algorytmach genetycznych iteracja jest równoznaczna z utworzeniem nowej generacji.

Literatura

- Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.
- Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.
- Mitchell, M. (1996). *An introduction to genetic algorithms*. MIT Press.
- Bäck, T. (1996). *Evolutionary algorithms in theory and practice: Evolution strategies, evolutionary programming, genetic algorithms*. Oxford University Press.
- Michalewicz, Z. (1996). *Genetic algorithms + data structures = evolution programs* (3rd ed.). Springer.
- De Jong, K. A. (2006). *Evolutionary computation: A unified approach*. MIT Press.
- Rechenberg, I. (1973). *Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*. Frommann-Holzboog.
- Schwefel, H.-P. (1981). *Numerical optimization of computer models*. Wiley.
- Eiben, A. E., & Smith, J. E. (2003). *Introduction to evolutionary computing*. Springer.
- Molga, M., & Smutnicki, C. (2005). *Test functions for optimization needs* (Technical Report). Wrocław University of Technology.